

# FitTrack AI

*Personalized fitness recommendations powered by AI*

## Project Documentation

### Team 4

Team ID : SWTID-2026-6232

Project Name : FitTrack AI — AI-Powered Fitness Tracking Backend API

| Name            | Email                       |
|-----------------|-----------------------------|
| John Joseph I   | Jovinooviyam@gmail.com      |
| Kamala kannan V | K7965223@gmail.com          |
| Kanniyappan S   | 124csbkanniyappan@gmail.com |
| Kamesh M        | Manidevi859@gmail.com       |

## Description

The FitTrack AI backend is a secure, scalable, and AI-powered RESTful application developed to simplify workout tracking and provide intelligent fitness recommendations. Built using Node.js, Express.js, MongoDB, and Mongoose, the backend exposes REST APIs that enable secure user authentication, workout management, workout searching, and AI-assisted fitness guidance.

The application follows the Model-View-Controller (MVC) architecture, making the project modular, maintainable, and scalable. User authentication is implemented using JSON Web Tokens (JWT), while passwords are securely encrypted using bcrypt.js before storage in MongoDB. Every protected API validates the JWT token through middleware before allowing access to sensitive resources.

A key feature of the project is its integration with Google Gemini AI, which provides personalized workout recommendations based on a user's age, fitness goal, and experience level, and generates fitness insights from workout statistics such as total workouts, average duration, and calories burned.

## Scenario-Based Case Study

### Background

Rahul is a fitness enthusiast who exercises regularly but records his workouts manually using notebooks and spreadsheets. He finds it difficult to organize workout history, monitor progress, and determine whether his exercise habits align with his fitness goals, and lacks professional guidance on which workouts suit his current fitness level.

### Problem

- Difficulty maintaining a centralized record of workouts.
- Manual tracking leads to inaccurate workout history.
- No secure user authentication for personal fitness records.
- Unable to search previous workouts efficiently.
- No personalized workout recommendations.
- No insights regarding overall workout performance.
- Time-consuming manual calculation of calories burned and workout duration.

### Solution

FitTrack AI provides a centralized backend where users can securely register, authenticate, and manage workout records through RESTful APIs. By integrating Google Gemini AI, the application analyzes user information and workout statistics to generate intelligent recommendations and personalized insights, backed by JWT authentication, password hashing, middleware-based authorization, centralized error handling, and MongoDB schema validation.

## Usage

### User Usage

A registered user creates an account and logs in with email and password. The backend issues a JWT token that authorizes access to protected APIs, which let the user:

- Create, view, update, delete, and search workout records.
- Retrieve a workout by its unique identifier.
- Generate AI-powered workout recommendations.
- Obtain AI-generated fitness insights from workout statistics.

### AI Usage

Users submit age, fitness goal, and experience level; Gemini returns a personalized workout plan, weekly exercise suggestions, level-appropriate recommendations, motivational guidance, and safety tips. Users also submit total workouts, average duration, and total calories burned; the AI module returns personalized fitness insights.

## Outcome

FitTrack AI improves fitness tracking by eliminating manual record keeping and providing intelligent, activity-based recommendations. Secure authentication ensures only authorized users access personal workout data, and the modular MVC architecture enables future extensions such as nutrition tracking, BMI calculation, wearable integration, and analytics dashboards.

## 1. Software Requirements

- Operating System: Windows 10/11, macOS, or Linux
- Node.js (v16 or above)
- npm (v8 or above)
- Express.js
- MongoDB
- Postman / Thunder Client for API verification
- Code Editor: Visual Studio Code

## 2. Hardware Requirements

- Processor: Intel Core i5 (8th Gen+) / AMD Ryzen 5 or equivalent
- RAM: 8 GB minimum (16 GB recommended)
- Storage: 1 GB of available disk workspace

## Epic 1: Project Architecture

FitTrack AI follows a layered architecture based on the MVC design pattern, separating routing, business logic, database interaction, middleware, and AI services into independent modules.

When a client sends an HTTP request, the Express server forwards it to the corresponding route. Protected routes pass through JWT authentication middleware before reaching controllers, which invoke business logic or AI services as needed. Mongoose models handle database interaction, and Google Gemini AI is accessed through a dedicated service layer. The controller returns a structured JSON response to the client.

## Overall Architecture Components

- Client Application (React / Mobile App / Postman)
- Express Server
- API Routes
- Authentication Middleware
- Controllers
- Services
- Google Gemini AI Service
- MongoDB Database
- Mongoose Models

## Components Definition

## 1. Client Layer

Entry point of the application (React app, mobile app, or Postman). Sends API requests, stores the JWT token, invokes AI recommendation APIs, and displays responses.

## 2. Express Server Gateway

Starts the HTTP server, parses JSON, enables CORS, registers routes, connects middleware, and handles centralized errors.

## 3. Authentication Middleware

Validates the JWT on every protected request: reads the Authorization header, verifies the token signature, decodes user information, attaches it to the request, and rejects unauthorized calls.

## 4. Route Layer

| Group          | Endpoints                                       |
|----------------|-------------------------------------------------|
| Authentication | Register, Login, Profile                        |
| Workout        | Add, Get All, Search, Get by ID, Update, Delete |
| AI             | Workout Recommendation, Fitness Insights        |

## 5. Controller Layer

Contains business logic: the Authentication Controller handles registration, login, and profile retrieval; the Workout Controller handles CRUD and search; the AI Controller sends prompts to Gemini and returns AI-generated responses.

## 6. Service Layer

- Gemini AI Service — prepares prompts, calls Gemini, returns formatted recommendations.
- JWT Service — creates tokens, configures expiry, verifies identity.
- Password Service — hashes passwords and compares them at login.

## 7. Database Layer

MongoDB stores all persistent data via Mongoose collections for Users and Workouts, offering a flexible schema, high performance, and easy retrieval.

# Entity Relationship (ER) Diagram Description

## 1. User Entity

| Field     | Type     | Description          |
|-----------|----------|----------------------|
| id        | ObjectId | Primary Key          |
| name      | String   | User Name            |
| email     | String   | Unique Email Address |
| password  | String   | Encrypted Password   |
| createdAt | Date     | Record Creation Time |
| updatedAt | Date     | Last Updated Time    |

## 2. Workout Entity

| Field       | Type     | Description                |
|-------------|----------|----------------------------|
| id          | ObjectId | Primary Key                |
| user        | ObjectId | References User Collection |
| workoutName | String   | Workout Name               |

|                |        |                    |
|----------------|--------|--------------------|
| category       | String | Workout Category   |
| duration       | Number | Duration (Minutes) |
| caloriesBurned | Number | Calories Burned    |
| workoutDate    | Date   | Workout Date       |
| createdAt      | Date   | Created Timestamp  |
| updatedAt      | Date   | Updated Timestamp  |

Relationship: User → Workout

One-to-Many. A single user can create multiple workout records; each workout belongs to exactly one user, maintained via the user field (ObjectId) inside the Workout schema.

Key Features

1. Secure Authentication

JWT-based authentication with user registration, secure login, password encryption, token generation, and protected APIs.

2. Workout Management

Full CRUD: add, view, update, and delete workouts, each linked to its authenticated user.

3. Workout Search

Search by workout name, category, or date.

4. Google Gemini Integration

Input: age, fitness goal, experience level. Output: weekly workout plan, suggested exercises, training tips, safety recommendations.

5. AI Fitness Insights

Input: total workouts, average duration, calories burned. Output: performance analysis, improvement suggestions, motivational advice, and a progress summary.

6. Centralized Error Handling

Meaningful JSON error responses for invalid credentials, missing tokens, validation errors, database errors, and internal server errors.

7. RESTful API Design

Standard HTTP methods (GET, POST, PUT, DELETE) with JSON responses throughout.

8. Modular Architecture

Responsibilities separated into Models, Controllers, Routes, Middleware, Services, and Configuration.

Roles and Responsibilities

Authenticated User

| Operation | Permission |
|-----------|------------|
| Register  | Yes        |
| Login     | Yes        |

|                   |     |
|-------------------|-----|
| View Profile      | Yes |
| Add Workout       | Yes |
| Get Workouts      | Yes |
| Search Workouts   | Yes |
| Update Workout    | Yes |
| Delete Workout    | Yes |
| AI Recommendation | Yes |
| Fitness Insights  | Yes |

## MVC Architecture Pattern

### Model Layer

Defines data structure via Mongoose schemas (User, Workout); handles schema definition, data validation, CRUD operations, and collection mapping.

### View Layer

As a backend REST API, the View Layer is represented by API consumers (React app, mobile app, Postman) sending requests and receiving JSON responses.

### Controller Layer

Intermediary between View and Model: request validation, business logic execution, database interaction, AI service invocation, and response generation.

## Epic 2: Project Setup and Configuration

The backend is built with Node.js and Express.js, MongoDB as the primary database, and Google Gemini AI for recommendations.

### Creating the Project

- Create the project folder: AI-FitTrack-API
- Open it in Visual Studio Code
- Create a server folder for backend source code
- Run `npm init -y` to generate package.json
- Install dependencies: `npm install express mongoose dotenv bcryptjs jsonwebtoken cors`
- Install nodemon as a dev dependency: `npm install --save-dev nodemon`

### Project Folder Structure

```
code
├── node_modules
├── src
│   ├── config
│   │   └── db.js
│   ├── controllers
│   │   ├── aiController.js
│   │   ├── authController.js
│   │   └── workoutController.js
│   ├── middleware
│   │   ├── auth.js
│   │   └── errorHandler.js
│   └── models
│       ├── User.js
│       └── Workout.js
```

```

├── routes
│   ├── aiRoutes.js
│   ├── authRoutes.js
│   ├── index.js
│   └── workoutRoutes.js
├── services
│   └── geminiService.js
├── app.js
├── server.js
├── .env
├── .env.example
├── .gitignore
├── FitTrack.postman_collection.json
├── package-lock.json
├── package.json
└── README.md

```

## Environment Variables (.env)

- PORT=5000
- MONGO\_URI=mongodb://localhost:27017/aifittrack
- JWT\_SECRET=your\_secret\_key
- GEMINI\_API\_KEY=your\_google\_gemini\_api\_key

## Epic 3: Backend Development

Backend development implements REST APIs across server configuration, database configuration, authentication, workout management, AI recommendation, middleware, and error handling.

## Epic 4: Database Configuration

MongoDB is the primary NoSQL database; Mongoose provides schema validation, middleware support, relationships, and simplified CRUD. The connection is configured in config/db.js, which connects to MongoDB, handles connection errors, displays connection status, and exports the database connection.

### 1. Database Connection (config/db.js)

Establishes the MongoDB connection using Mongoose, logs the connected host on success, and exits the process if the connection fails.

```

const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error connecting to MongoDB: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;

```

### 2. User Schema (models/User.js)

- Name cannot be empty.
- Email must be unique and valid.

- Password is mandatory and encrypted with bcrypt.js before saving.
- Automatic timestamps are enabled.

### 3. Workout Schema (models/Workout.js)

- Workout name and category are mandatory.
- Duration must be greater than zero.
- Calories burned must be greater than or equal to zero.
- Workout date is required.
- Every workout belongs to one authenticated user.

### 4. User.js – Full Source Code

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Please add a name'],
      trim: true,
    },
    email: {
      type: String,
      required: [true, 'Please add an email'],
      unique: true,
      trim: true,
      lowercase: true,
      match: [
        /^\.w+([\.-]?\w+)*\.w+([\.-]?\w+)*(\.w{2,3})+$/ ,
        'Please add a valid email address',
      ],
    },
    password: {
      type: String,
      required: [true, 'Please add a password'],
      minlength: [6, 'Password must be at least 6 characters'],
    },
  },
  {
    timestamps: true,
  }
);

// Pre-save password hashing hook
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) {
    return next();
  }
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

// Compare password method
userSchema.methods.matchPassword = async function (enteredPassword) {
  return await bcrypt.compare(enteredPassword, this.password);
};

module.exports = mongoose.model('User', userSchema);
```

## 5. Workout.js – Full Source Code

```
const mongoose = require('mongoose');

const workoutSchema = new mongoose.Schema(
  {
    workoutName: {
      type: String,
      required: [true, 'Please add a workout name'],
      trim: true,
    },
    category: {
      type: String,
      required: [true, 'Please select a workout category'],
      enum: {
        values: [
          'Cardio',
          'Strength Training',
          'Yoga',
          'Running',
          'Cycling',
          'Walking',
        ],
        message: '{VALUE} is not a supported workout category',
      },
    },
    duration: {
      type: Number,
      required: [true, 'Please add the workout duration in minutes'],
      min: [1, 'Duration must be at least 1 minute'],
    },
    caloriesBurned: {
      type: Number,
      required: [true, 'Please specify calories burned'],
      min: [0, 'Calories burned cannot be negative'],
    },
    workoutDate: {
      type: Date,
      default: Date.now,
    },
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true,
    },
  },
  {
    timestamps: true,
  }
);

module.exports = mongoose.model('Workout', workoutSchema);
```

## Epic 5: Project Execution & API Testing

The server is started with `npm start` and tested end-to-end using Thunder Client / Postman, in the order: Authentication APIs, Workout APIs, then AI APIs. All requests below were exercised against `http://localhost:5000` and returned the status codes shown.

### Program Execution

The backend is started in the terminal with `npm i` to install dependencies, followed by `npm start`. On a successful run, the server logs its listening port and confirms the MongoDB connection before it is ready to accept requests.

```

problems Output Terminal Ports
Terminal
PS D:\fittrack api\code> npm i

up to date, audited 179 packages in 2s

28 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
npm warn allow-scripts 2 packages have install scripts not yet covered by allowScripts:
npm warn allow-scripts @google/genai@2.24.0 (install: (install scripts present))
npm warn allow-scripts protobufjs@7.6.6 (install: (install scripts present))
npm warn allow-scripts Run `npm approve-scripts --allow-scripts-pending` to review, or `npm approve-scripts <pkg>` to allow.
PS D:\fittrack api\code> npm start

> ai-fittrack-api@1.0.0 start
> node src/server.js

Server running in development mode on port 5000
MongoDB Connected: ac-u6mlxor-shard-00-00.q22e1vw.mongodb.net

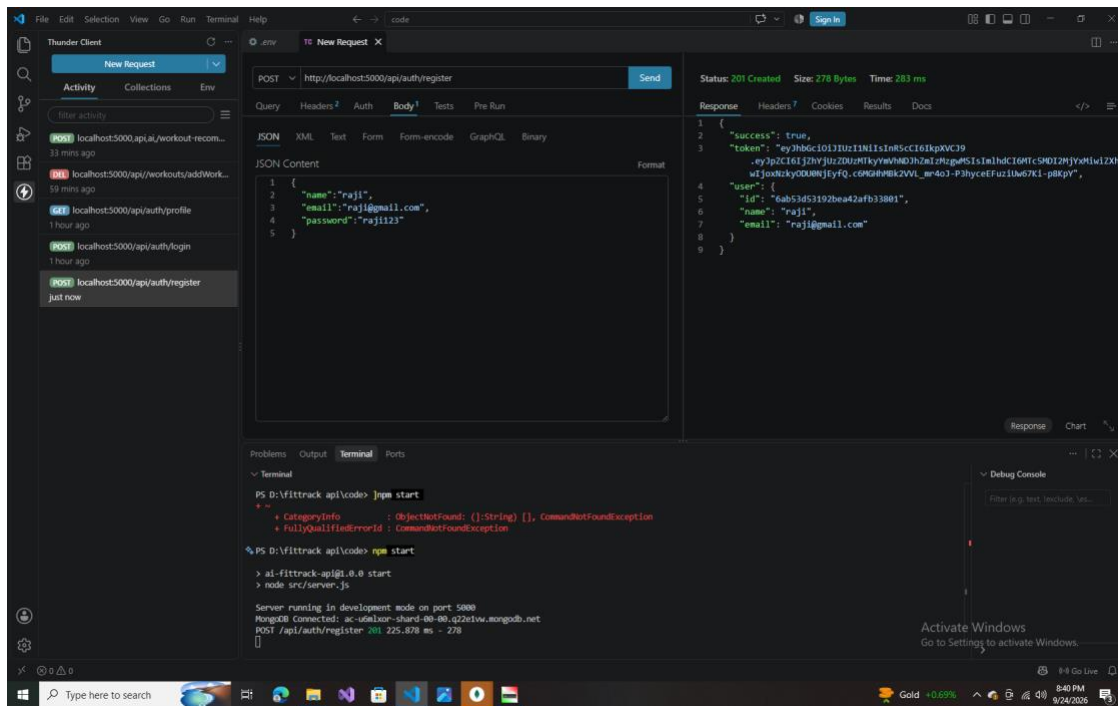
```

Terminal output — `npm i`, `npm start`, server running on port 5000, MongoDB Connected

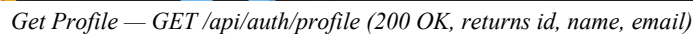
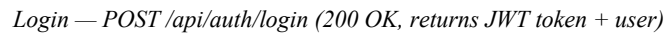
## Authentication API Testing

| Endpoint           | Method | Sample Request Body                        | Result                            |
|--------------------|--------|--------------------------------------------|-----------------------------------|
| /api/auth/register | POST   | name, email, password                      | 201 Created — returns user object |
| /api/auth/login    | POST   | email: selva@gmail.com, password: selva@01 | 200 OK — returns JWT token + user |
| /api/auth/profile  | GET    | Authorization: Bearer <token>              | 200 OK — returns id, name, email  |

Note: calling `/api/auth/profile` without a valid token returns 401 Unauthorized with a “JWT verification error: invalid token” message, confirming the middleware correctly rejects unauthorized access.



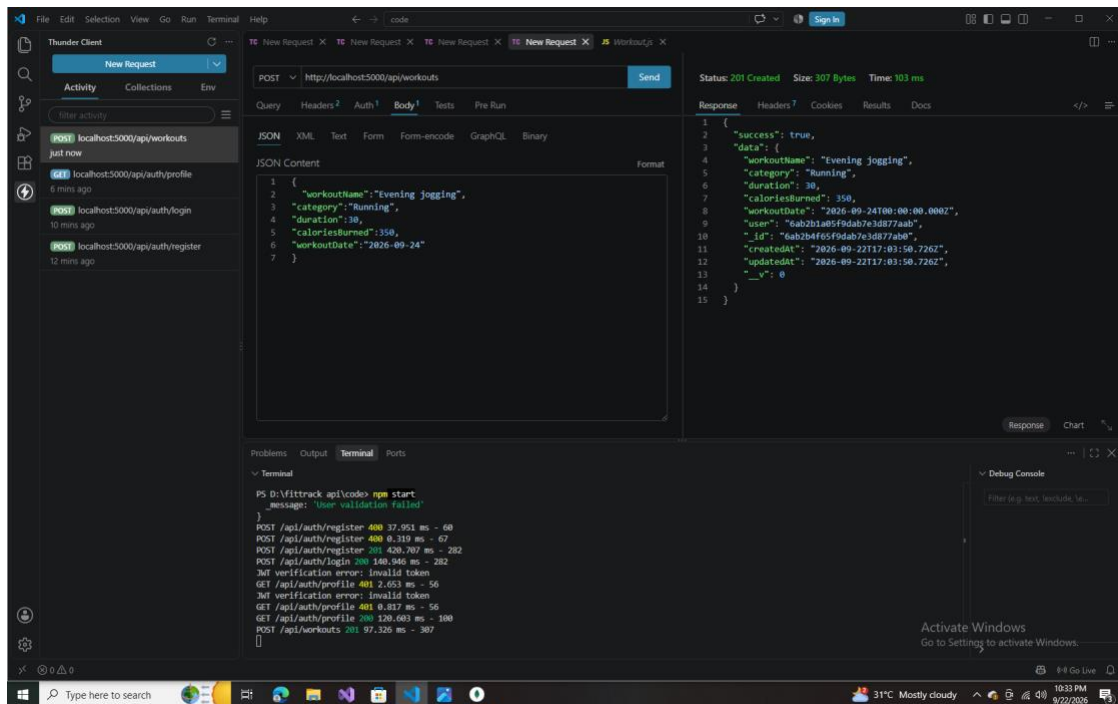
Register — `POST /api/auth/register` (201 Created, returns user object + token)



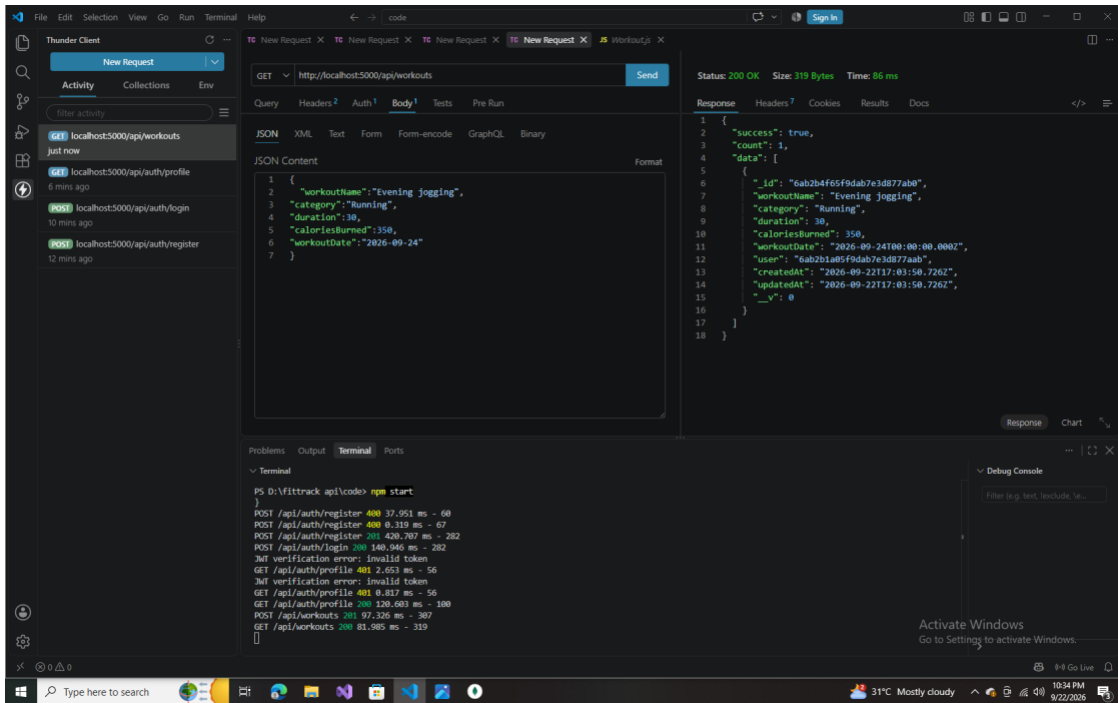
| Endpoint      | Method | Sample Request Body                                          | Result                              |
|---------------|--------|--------------------------------------------------------------|-------------------------------------|
| /api/workouts | POST   | workoutName, category, duration, caloriesBurned, workoutDate | 201 Created — returns saved workout |

|                   |        |                                                |                                         |
|-------------------|--------|------------------------------------------------|-----------------------------------------|
| /api/workouts     | GET    | —                                              | 200 OK — returns count + data array     |
| /api/workouts/:id | GET    | —                                              | 200 OK — returns single workout         |
| /api/workouts/:id | PUT    | updated fields (e.g. duration, caloriesBurned) | 200 OK — returns updated workout        |
| /api/workouts/:id | DELETE | —                                              | 200 OK — “Workout removed successfully” |

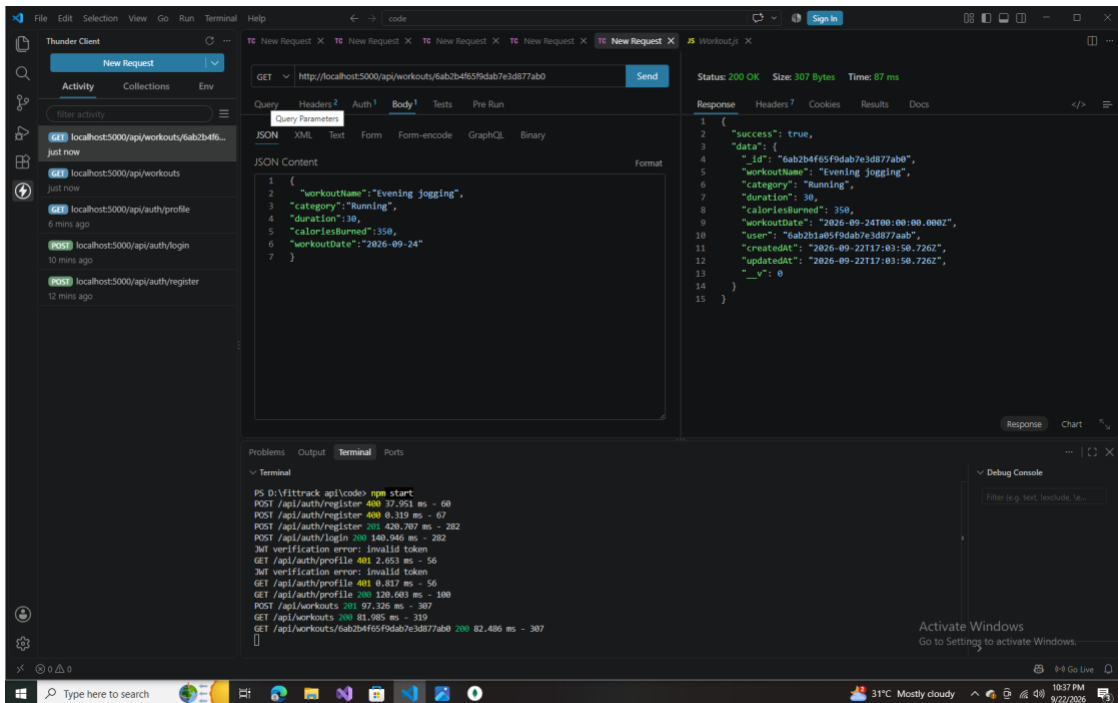
Sample workout used across the Add / Get / Update / Delete tests: workoutName “Evening jogging”, category “Running”, duration 30–40 minutes, caloriesBurned 100–350, workoutDate 2026-09-24.



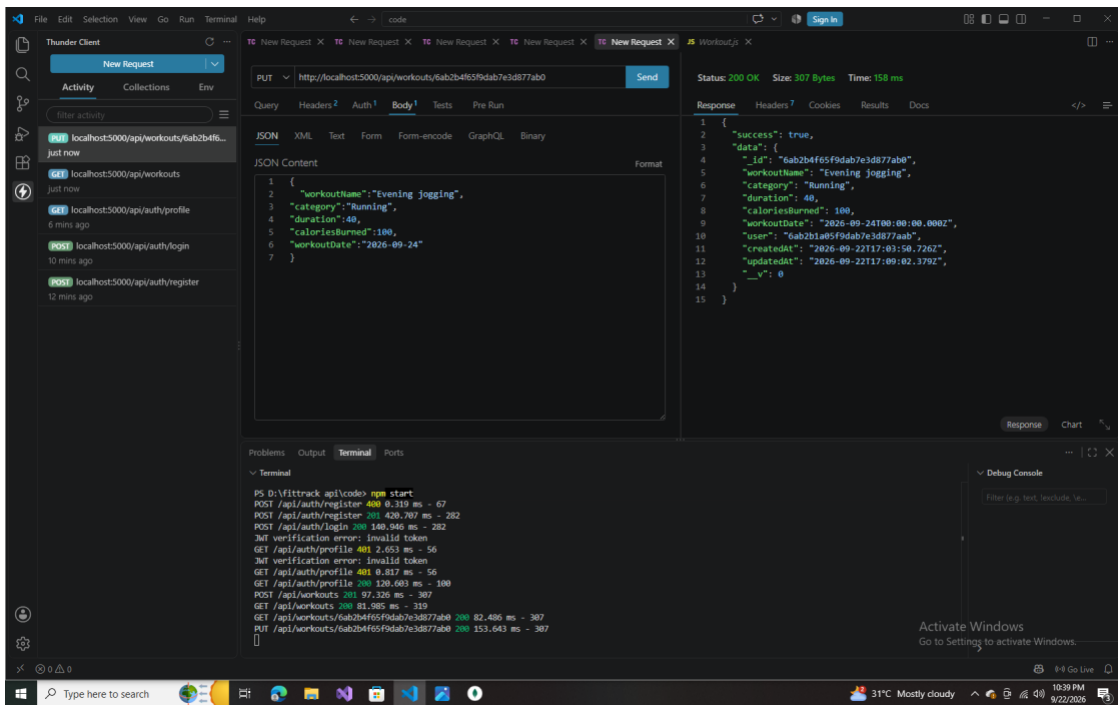
Add Workout — POST /api/workouts (201 Created)



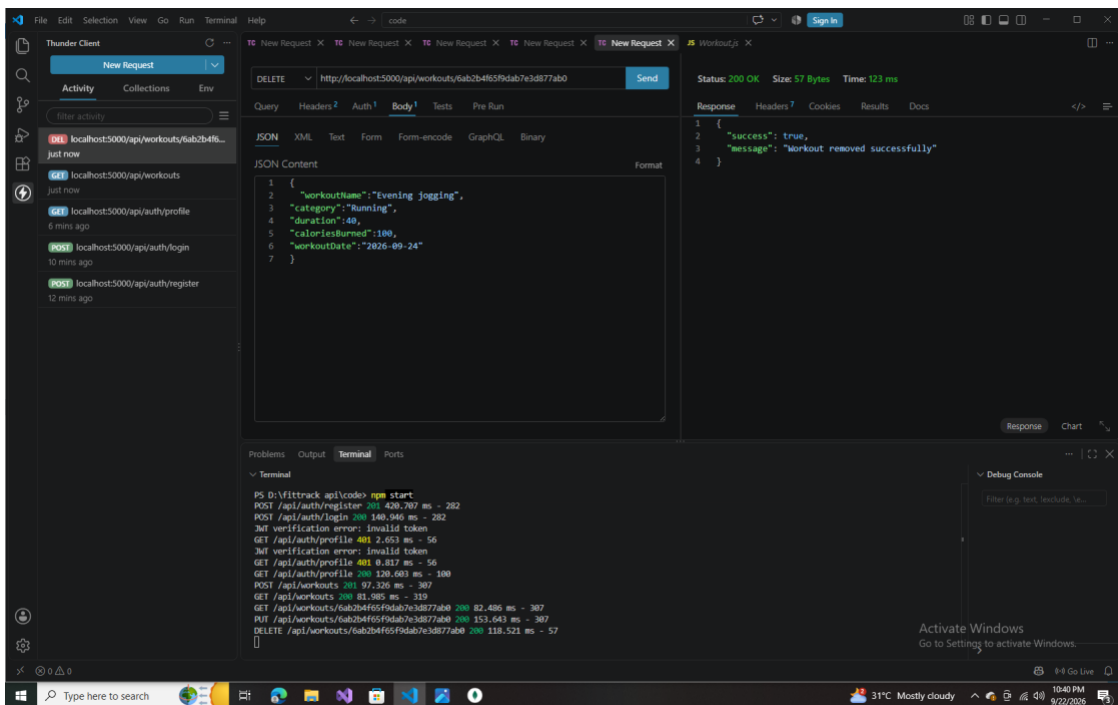
Get All Workouts — GET /api/workouts (200 OK)



Get Workout by ID — GET /api/workouts/:id (200 OK)



Update Workout — PUT /api/workouts/:id (200 OK)



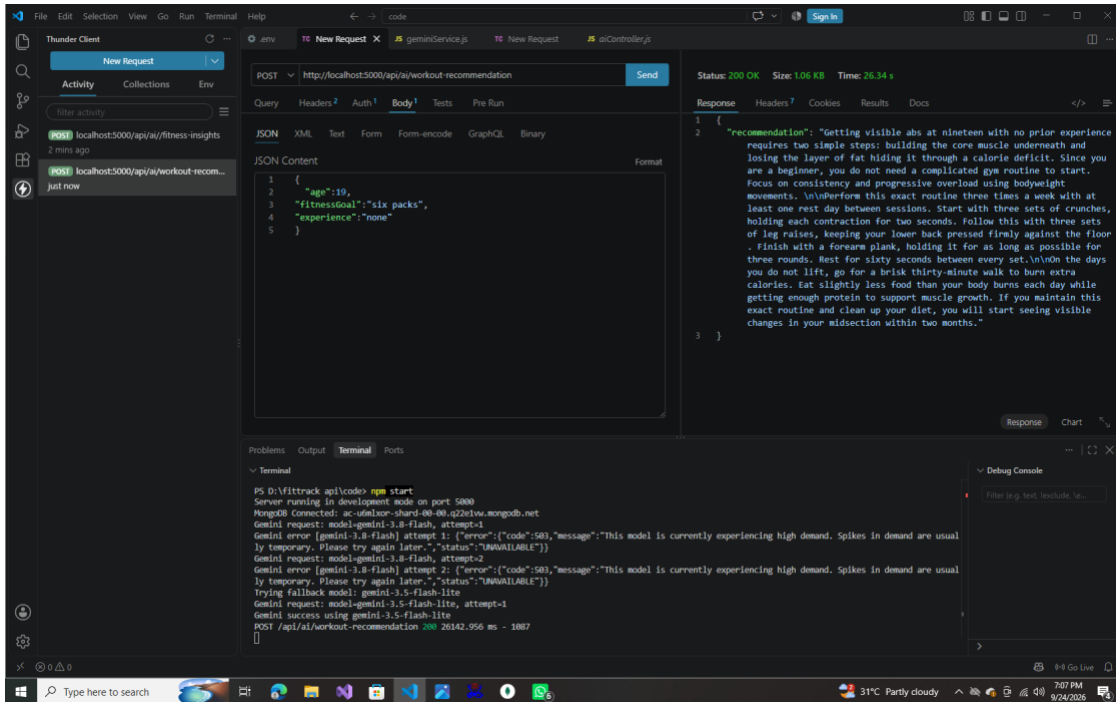
Delete Workout — DELETE /api/workouts/:id (200 OK)

## AI API Testing

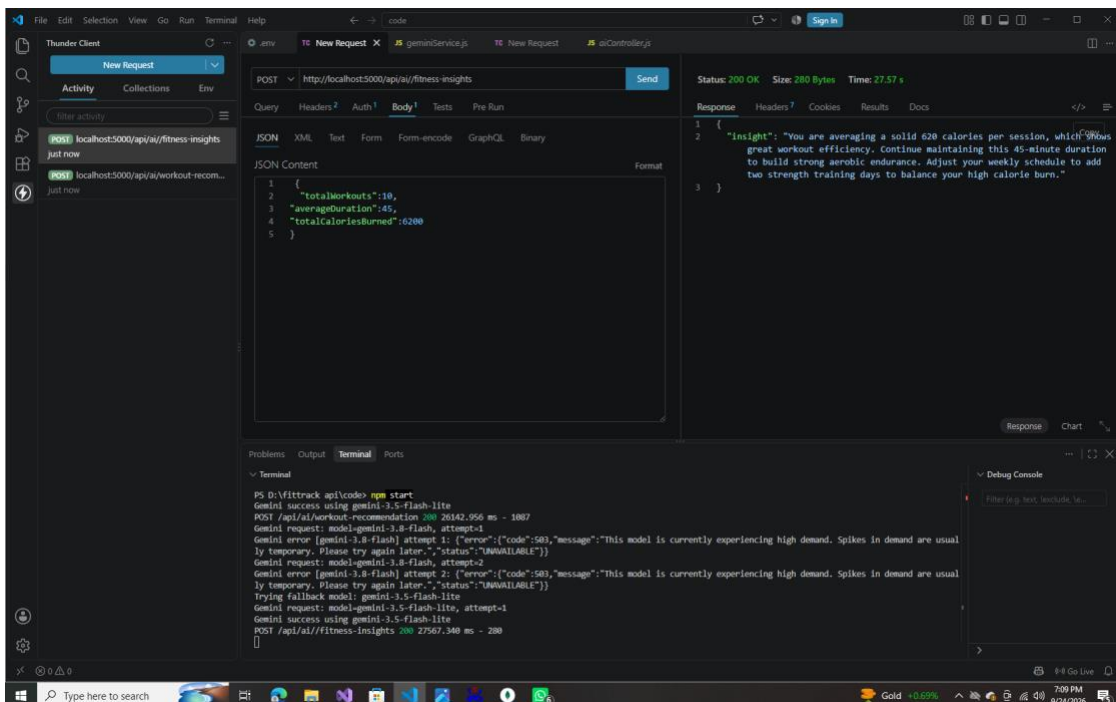
| Endpoint                       | Method | Sample Request Body                                   | Result                                                 |
|--------------------------------|--------|-------------------------------------------------------|--------------------------------------------------------|
| /api/ai/workout-recommendation | POST   | age: 19, fitnessGoal: “six packs”, experience: “none” | 200 OK — returns a personalized workout recommendation |

|                          |      |                                                                   |                                                 |
|--------------------------|------|-------------------------------------------------------------------|-------------------------------------------------|
| /api/ai/fitness-insights | POST | totalWorkouts: 10, averageDuration: 45, totalCaloriesBurned: 6200 | 200 OK — returns a personalized fitness insight |
|--------------------------|------|-------------------------------------------------------------------|-------------------------------------------------|

Server logs during AI testing show the primary model (gemini-3.8-flash) returning a 503 “currently experiencing high demand” error on the first two attempts; the service automatically retried and fell back to gemini-3.5-flash-lite, which completed both requests successfully. This confirms the AI service layer's retry-and-fallback logic is working as intended.



*Workout Recommendation — POST /api/ai/workout-recommendation (200 OK)*



*Fitness Insights — POST /api/ai/fitness-insights (200 OK)*

## Testing Order

- Authentication APIs (Register → Login → Profile)
- Workout APIs (Add → Get All → Get by ID → Update → Delete)
- AI APIs (Workout Recommendation → Fitness Insights)

## Conclusion

FitTrack AI successfully delivers a secure, modular, and AI-powered backend for fitness tracking. The layered MVC architecture keeps routing, business logic, database access, and AI integration cleanly separated, while JWT authentication and bcrypt password hashing protect user data throughout. End-to-end testing across the authentication, workout, and AI modules confirmed that every endpoint behaves as designed — including graceful handling of invalid tokens, validation errors, and the automatic model fallback when the primary Gemini model is unavailable.

Beyond meeting its core goal of replacing manual, notebook-based workout tracking, the project establishes a solid, extensible foundation. Its modular design means features such as nutrition tracking, BMI calculation, wearable device integration, workout reminders, and analytics dashboards can be added later without restructuring the existing codebase.

## Project Links

| Resource                  | Link                                                                                                                                                                                                  |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Demo Video                | <a href="https://drive.google.com/file/d/1ZK6iTGvzySWMYtz1RjakBr3ikD5gD8Fh/view?usp=sharing">https://drive.google.com/file/d/1ZK6iTGvzySWMYtz1RjakBr3ikD5gD8Fh/view?usp=sharing</a>                   |
| API Testing Video         | <a href="https://drive.google.com/file/d/1ClxF-sCbgSfLgH-88dKkQq5efrqgMGsC/view?usp=sharing">https://drive.google.com/file/d/1ClxF-sCbgSfLgH-88dKkQq5efrqgMGsC/view?usp=sharing</a>                   |
| Code (Repository / Drive) | <a href="https://github.com/jovinooviyam-crypto/Ai-Argmented-Backend-Development-Ai-FitTrack-API.git">https://github.com/jovinooviyam-crypto/Ai-Argmented-Backend-Development-Ai-FitTrack-API.git</a> |